⍰ Stream API Execution Order in Java 1.8

In **Java 8**, the Stream API processes operations in a specific way that often confuses developers.

✅ Short Answer:

- **Stream operations are written left → right**
- **But execution happens element-by-element (vertically), not step-by-step horizontally**
- **Terminal operation triggers execution**
- Processing follows **pipeline order (left → right)** for each element

---

□ Important Concept: Horizontal vs Vertical Execution

❌ What Many Think (Horizontal Execution)

filter → filter → map → collect
(All elements complete filter first, then next filter...)

✅ What Actually Happens (Vertical Execution)

Element 1 → filter → filter → map
Element 2 → filter → filter → map
Element 3 → filter → filter → map

□ Java processes **one element at a time through the entire pipeline**

□ Example 1: Understanding Execution Order

```
import java.util.Arrays;
import java.util.List;

public class StreamExecutionDemo {
   public static void main(String[] args) {

      List<String> names = Arrays.asList("Amit", "Rahul", "Ankit", "Ravi");

      names.stream()
            .filter(name -> {
               System.out.println("Filter 1: " + name);
               return name.startsWith("A");
            })
            .filter(name -> {
               System.out.println("Filter 2: " + name);
               return name.length() > 4;
            })
            .map(name -> {
               System.out.println("Map: " + name);
               return name.toUpperCase();
            })
            .forEach(name -> System.out.println("Final Output: " + name));
   }
}
```

□ Step-by-Step Execution Explanation

Input List:

Amit, Rahul, Ankit, Ravi

Execution Flow:

□ Element 1: "Amit"

1. Filter 1 → passes (starts with A)
2. Filter 2 → fails (length 4)
   ✕ Stops here (map not executed)

□ Element 2: "Rahul"

1. Filter 1 → fails
   ✕ Stops immediately

□ Element 3: "Ankit"

1. Filter 1 → passes
2. Filter 2 → passes
3. Map → converts to ANKIT
4. forEach → prints ANKIT

□ Element 4: "Ravi"

1. Filter 1 → fails
   ❌ Stops

□ Output Order

Filter 1: Amit
Filter 2: Amit
Filter 1: Rahul
Filter 1: Ankit
Filter 2: Ankit
Map: Ankit
Final Output: ANKIT
Filter 1: Ravi

□ Key Observations

Concept	Explanation
Execution Direction	Left → Right (pipeline order)
Processing Style	Vertical (element by element)
Lazy Evaluation	Yes (nothing runs until terminal operation)
Short-Circuiting	Yes (if filter fails, next ops skipped)

□ Example 2: Short-Circuit with findFirst()

```
names.stream()
    .filter(name -> {
        System.out.println("Filtering: " + name);
        return name.startsWith("A");
    })
    .findFirst()
    .ifPresent(System.out::println);
```

□ What Happens?

- "Amit" → passes filter
- findFirst() stops processing immediately
- "Rahul", "Ankit", "Ravi" never checked

□ This is **short-circuiting behavior**

□ Visual Representation

Amit → F1 → F2 ❌
Rahul → F1 ❌
Ankit → F1 → F2 → Map → Output
Ravi → F1 ❌

□ Final Conclusion

✔ Stream operations are **declared left to right**

✔ Execution happens **left to right per element**

✔ Processing is **vertical (one element at a time)**

✔ Terminal operation triggers execution

✔ Supports short-circuiting

If you want, I can also show:

- ✅ Parallel Stream execution order
- ✅ Internal working with Spliterator
- ✅ Bytecode-level explanation
- ✅ Interview-ready explanation format
- ✅ UML-style execution diagram

✅ 1 Parallel Stream Execution Order (Java 8)

□ What Changes in Parallel Stream?

When you use:

list.parallelStream()

Java internally uses the **ForkJoinPool (common pool)** to split data and process elements in parallel.

□ Example

```
import java.util.Arrays;
import java.util.List;

public class ParallelDemo {
   public static void main(String[] args) {

      List<Integer> numbers = Arrays.asList(1,2,3,4,5,6);

      numbers.parallelStream()
            .filter(n -> {
               System.out.println(Thread.currentThread().getName() +
                              " Filter: " + n);
               return n % 2 == 0;
            })
            .map(n -> {
               System.out.println(Thread.currentThread().getName() +
                              " Map: " + n);
               return n * n;
            })
            .forEach(n -> System.out.println(
                  Thread.currentThread().getName() +
                  " Output: " + n));
   }
}
```

□ What Happens?

- Data is split into chunks
- Multiple threads process elements
- Order is **NOT guaranteed** with forEach
- Order is preserved with forEachOrdered

 Execution Model

Feature	Sequential	Parallel
Thread Used	Main thread	ForkJoinPool
Order Guarantee	Yes	No (unless ordered terminal)
Performance	Single-core	Multi-core
Processing	Vertical	Vertical per thread

✅ 2 Internal Working with Spliterator

 What is Spliterator?

Spliterator = **Split + Iterator**

It helps:

- Split data for parallel execution
- Traverse elements efficiently
- Support lazy evaluation

 How It Works Internally

When you call:

list.parallelStream()

Steps:

1. Collection.stream() creates a Stream
2. Stream gets a Spliterator
3. trySplit() divides data
4. ForkJoinPool assigns chunks to threads
5. Each thread processes pipeline vertically
6. Results are combined

□ Simplified Internal Flow

```
ArrayList
      ↓
Spliterator
      ↓ trySplit()
Chunk1     Chunk2
    ↓          ↓
Thread1    Thread2
    ↓          ↓
Pipeline   Pipeline
    ↓          ↓
 Combine Results
```

✅ 3 Bytecode-Level Explanation

Let's compile this:

```
numbers.stream()
    .filter(n -> n % 2 == 0)
    .map(n -> n * 2)
    .collect(Collectors.toList());
```

□ What Happens in Bytecode?

1. Lambda → Converted into **invokedynamic**
2. JVM uses **LambdaMetafactory**
3. Functional interfaces generated at runtime
4. Stream pipeline objects created
5. Terminal operation triggers evaluation

□ Lambda Bytecode Concept

Instead of anonymous class:

```
new Predicate<Integer>() {
   public boolean test(Integer n) {
      return n % 2 == 0;
   }
}
```

Java 8 generates:

```
invokedynamic
LambdaMetafactory.metafactory()
```

□ This makes lambdas lightweight & faster.

✅ 4 Interview-Ready Explanation Format

If interviewer asks:

❓ "How does Stream execution order work?"

□ Perfect Answer:

In Java 8, Stream operations are written left-to-right but executed vertically per element.
Intermediate operations are lazy and only execute when a terminal operation is invoked.
In sequential streams, elements are processed one-by-one in encounter order.
In parallel streams, the Spliterator splits data into chunks processed by ForkJoinPool threads, and order is not guaranteed unless explicitly requested.

❓ "What is Spliterator?"

Spliterator is an internal iterator introduced in Java 8 that supports parallel processing by splitting the data source into smaller parts and traversing elements efficiently.

❓ "Why are Streams lazy?"

Because intermediate operations do not execute immediately. They build a pipeline, and execution starts only when a terminal operation is invoked.

✅ 5 UML-Style Execution Diagram

□ Sequential Stream

```
+----------------+
| Data Source |
+----------------+
      |
      v
+-------------------+
| Stream Object |
+-------------------+
      |
      v
```

```
+----------+
| filter()  |
+----------+
     |
     v
+-----------+
| map()     |
+-----------+
     |
     v
+-----------+
| collect() |
+-----------+
     |
     v
  Final Result
```

□ Parallel Stream UML View

```
          +----------------+
          | Data Source |
          +----------------+
                  |
                  v
          +-----------------+
          |  Spliterator    |
          +-----------------+
              /           \
             /             \
     +--------------+   +---------------+
     | Thread-1   |    | Thread-2   |
     +--------------+   +---------------+
            |                  |
      filter/map        filter/map
            |                  |
            \                 /
             \               /
            +------------------+
            |  Combine      |
            +-----------------+
                    |
                Final Result
```

□ Final Advanced Summary

Concept	Explanation

Direction	Left → Right
Execution	Vertical per element
Lazy	Yes
Parallel	Uses Spliterator + ForkJoinPool
Order in Parallel	Not guaranteed
Lambda Implementation	Invokedynamic
Short Circuit	Supported

If you want next level deep dive:

- □ JDK internal class hierarchy (ReferencePipeline, AbstractPipeline)
- □ Source code walkthrough of Stream.java
- □ ForkJoinPool internal working
- □ Performance benchmarking sequential vs parallel
- □ Memory & GC behavior of streams

✅ How Java Uses ForkJoinPool (Common Pool) in Parallel Streams

When you write:

list.parallelStream()

Java internally uses:

ForkJoinPool.commonPool()

to split data and execute tasks in parallel.

□ 1 What is ForkJoinPool?

□ Introduced in **Java 7**
□ Used for parallel task execution
□ Based on **Work-Stealing Algorithm**

□ Core Idea

- **Fork** → Split task into smaller subtasks
- **Join** → Combine results

Parallel Streams internally use this mechanism.

□ 2 Simple Java Example (Thread Proof)

```
import java.util.Arrays;
import java.util.List;

public class ParallelStreamDemo {

   public static void main(String[] args) {

      List<Integer> numbers = Arrays.asList(1,2,3,4,5,6,7,8);

      numbers.parallelStream()
            .map(n -> {
               System.out.println(
                  "Number: " + n +
                  " processed by: " +
                  Thread.currentThread().getName()
               );
               return n * n;
            })
            .forEach(System.out::println);
   }
}
```

□ Sample Output (Will Vary)

```
Number: 5 processed by: ForkJoinPool.commonPool-worker-1
Number: 2 processed by: ForkJoinPool.commonPool-worker-3
Number: 7 processed by: main
Number: 1 processed by: ForkJoinPool.commonPool-worker-2
```

□ Notice thread names:

ForkJoinPool.commonPool-worker-X

That proves Java is using the **common ForkJoinPool**

□ 3 Internally What Happens?

When you call:

numbers.parallelStream()

Step-by-step internal flow:

1. Collection.parallelStream() called
2. Creates a Stream with parallel flag = true
3. Gets a Spliterator
4. Spliterator calls trySplit()
5. Tasks submitted to:
6. ForkJoinPool.commonPool()
7. Each worker thread processes chunk
8. Results combined

□ 4 Show Common Pool Parallelism

```
import java.util.concurrent.ForkJoinPool;

public class CheckPool {
   public static void main(String[] args) {

      System.out.println(
         ForkJoinPool.commonPool().getParallelism()
      );
   }
}
```

Output Example:

7

□ Meaning:

AvailableProcessors - 1

If your CPU has 8 cores → parallelism = 7

□ 5 Manual ForkJoin Example (For Understanding)

This is what parallel stream roughly does internally:

```
import java.util.concurrent.RecursiveTask;
import java.util.concurrent.ForkJoinPool;

class SumTask extends RecursiveTask<Integer> {

   private int[] arr;
   private int start, end;

   public SumTask(int[] arr, int start, int end) {
      this.arr = arr;
      this.start = start;
      this.end = end;
   }

   @Override
   protected Integer compute() {

      if (end - start <= 2) {
         int sum = 0;
         for (int i = start; i < end; i++)
            sum += arr[i];
         return sum;
      }

      int mid = (start + end) / 2;

      SumTask left = new SumTask(arr, start, mid);
      SumTask right = new SumTask(arr, mid, end);

      left.fork();           // Fork
      int rightResult = right.compute();
      int leftResult = left.join();  // Join

      return leftResult + rightResult;
   }
}

public class ForkJoinExample {

   public static void main(String[] args) {

      int[] arr = {1,2,3,4,5,6,7,8};

      ForkJoinPool pool = ForkJoinPool.commonPool();

      int result = pool.invoke(new SumTask(arr, 0, arr.length));

      System.out.println("Sum = " + result);
```

```
    }
}
```

 This is the same **fork + split + join** logic used by parallel streams.

---

6 Internal Architecture Diagram

ForkJoinPool Execution Model

```
Data Source
      ↓
Spliterator
      ↓ trySplit()
 -----------------------------------
    |            |              |
Chunk1   Chunk2    Chunk3
    ↓            ↓              ↓
Worker1  Worker2   Worker3
    ↓            ↓              ↓
Pipeline  Pipeline  Pipeline
       \          |          /
        \         |         /
       Combine Results
```

---

7 Work-Stealing Explained

If:

- Worker1 finishes early
- Worker2 still has tasks

Worker1 will:

Steal remaining task from Worker2

This improves CPU utilization.

---

8 Important Points for Interviews

Question	Answer
Which pool does parallel stream use?	ForkJoinPool.commonPool()

Default parallelism?	CPU cores – 1
Can we change it?	Yes (Custom ForkJoinPool)
Order guaranteed?	No (unless forEachOrdered)
Internal splitting mechanism?	Spliterator

---

9 Custom ForkJoinPool with Parallel Stream

```
import java.util.concurrent.ForkJoinPool;
import java.util.Arrays;
import java.util.List;

public class CustomPoolDemo {

   public static void main(String[] args) throws Exception {

      ForkJoinPool customPool = new ForkJoinPool(4);

      customPool.submit(() ->
         Arrays.asList(1,2,3,4,5)
            .parallelStream()
            .forEach(n ->
               System.out.println(Thread.currentThread().getName())
            )
      ).get();

      customPool.shutdown();
   }
}
```

Now threads will be from your custom pool.

Final Concept Summary

Concept	Explanation
Parallel Stream	Uses ForkJoinPool
Pool Type	commonPool()
Splitting	Spliterator.trySplit()
Algorithm	Work-Stealing

Thread Type	ForkJoinWorkerThread
Trigger	Terminal operation

If you want next:

- ☐ Deep dive into Spliterator source code
- ☐ JDK ReferencePipeline internal classes
- ☐ Performance benchmark code
- ☐ Memory and GC behavior
- ☐ When NOT to use parallel stream